

AI技术应用验证报告

提交人：唐立峰
学号：2023141461045
课程：软件过程与管理
项目名称：面向网课的智能监督系统

一、验证背景

本次验证是在前两个文档《AI技术调研与场景分析》和《软件过程改进方案设计文档》的基础上继续完成的。在前面的分析中，本文已经识别出多个适合在“面向网课的智能监督系统”中引入AI技术的关键场景，包括AI辅助需求整理、AI辅助概要设计、AI辅助编码实现、AI辅助测试用例生成、AI辅助文档一致性检查等。

综合考虑落地难度、验证成本和对项目过程质量的提升效果，本报告选择“AI辅助测试用例生成”作为最小可行验证场景。选择该场景的原因主要有三点。

第一，测试验证是软件过程中的关键环节。对于课程项目来说，系统能否正常运行固然重要，但更重要的是能否在异常输入、边界条件和模块集成情况下保持稳定。如果测试用例设计不充分，系统在最终展示或答辩时就可能出现问题。

第二，测试用例生成任务比较适合AI辅助。测试用例通常具有较固定的结构，包括测试编号、测试目标、输入条件、执行步骤、预期结果和优先级。AI可以根据需求说明和接口字段快速生成表格化测试用例，适合与人工方式进行对比。

第三，该场景容易量化验证。相比AI辅助需求分析或AI辅助概要设计，测试用例生成可以通过耗时、测试用例数量、异常场景覆盖数量、人工修改比例等指标进行评价。因此，它适合作为本次作业中的原型验证场景。

二、验证目标

本次验证的目标是通过对比传统人工方式和AI辅助方式，判断AI技术是否能够在测试用例生成环节提升效率和质量。具体验证目标如下：

- 验证AI是否能够根据模块说明和接口字段生成结构化测试用例。
- 对比传统人工方式与AI辅助方式在测试用例设计耗时上的差异。
- 对比两种方式生成的测试用例数量和覆盖场景数量。
- 分析AI生成测试用例的可用性、准确性和需要人工修改的部分。
- 总结AI辅助测试用例生成在本项目中的应用价值和局限性。
- 为后续改进软件过程提供依据。

三、验证对象

本次验证对象为“面向网课的智能监督系统”中的视频采集与预处理模块。该模块是系统前端数据处理链路中的基础模块，负责从摄像头或视频文件中获取图像帧，对图像进行基础处理，并向后续特征提取模块提供统一格式的数据。

该模块的主要职责包括：

- 读取摄像头或视频流；
- 获取当前图像帧；
- 生成当前帧对应的时间戳；
- 对图像帧进行基础预处理；
- 整理人脸区域信息；
- 输出统一格式的数据结构；
- 对异常输入进行处理，例如摄像头不可用、视频帧为空、无人脸等。

模块输出字段设计如下：

字段名	类型	含义	异常情况
timestamp	float	当前帧时间戳	无法获取时记录异常状态或当前系统时间
frame	ndarray	当前图像帧	空帧时返回错误状态
faces	list	人脸检测结果列表	无人脸时为空列表
face_id	int / str	人脸编号	无法识别时为空
face_roi	tuple / list	人脸区域坐标	越界时需要修正或丢弃
owner_face_id	int / str / None	主体人脸编号	无法确定主体时为None

选择该模块进行验证，是因为它具有比较明确的输入输出，同时存在较多正常场景、异常场景和边界场景，适合测试用例设计。

四、验证工具与方法

4.1 AI工具选择

本次验证选择通用大语言模型工具作为AI辅助工具，例如ChatGPT、Kimi或通义千问。这里以ChatGPT作为主要验证工具进行说明。

选择该工具的原因如下：

1. 能够理解自然语言形式的项目背景和模块说明；
2. 能够根据接口字段生成结构化测试用例表；
3. 能够补充异常场景和边界场景；
4. 使用成本低，不需要额外搭建复杂环境；
5. 适合课程项目中的最小可行验证。

需要说明的是，本次验证的重点不是测试某一个AI工具本身的性能，而是验证“AI辅助测试用例生成”这一改进场景是否能够融入本项目的软件过程。

4.2 验证方法

本次验证采用对比实验方式。具体做法是：先采用传统人工方式设计测试用例，再采用AI辅助方式生成测试用例，然后从耗时、用例数量、场景覆盖、可执行性和人工修改情况等方面进行对比。

验证分为两个部分：

1. 传统人工方式

由本人根据视频采集与预处理模块的功能和接口字段，手动设计测试用例，并记录耗时和结果。

2. AI辅助方式

将模块说明、接口字段和测试要求输入AI工具，让AI生成测试用例表，再由人工进行筛选、修改和确认，并记录耗时和结果。

五、验证输入

为了保证AI辅助方式具有可复现性，本次验证设计了明确的输入提示词。输入内容如下：

请根据以下模块说明生成测试用例表。

项目名称：面向网课的智能监督系统。

当前模块：视频采集与预处理模块。

模块功能：

- 1. 从摄像头或视频流中读取图像帧；
- 2. 对图像帧进行基础预处理；
- 3. 整理人脸区域信息；
- 4. 输出统一格式的数据结构，供后续特征提取模块使用；
- 5. 对摄像头不可用、空帧、无人脸、多人脸等异常或边界情况进行处理。

模块输出字段：

timestamp: 当前帧时间戳，**float**类型；
frame: 当前图像帧，**ndarray**类型；
faces: 人脸检测结果列表，**list**类型；
face_id: 人脸编号，**int**或**str**类型；
face_roi: 人脸区域坐标，**tuple**或**list**类型；
owner_face_id: 主体人脸编号，可能为**int**、**str**或**None**。

请生成测试用例表，要求覆盖正常场景、异常场景、边界场景和性能场景。
表格字段包括：测试编号、测试目标、输入条件、执行步骤、预期结果、优先级。

该提示词包含了项目背景、模块功能、接口字段和测试要求，能够让AI围绕当前模块生成相对明确的测试用例。

六、传统人工方式验证过程与结果

6.1 人工设计过程

在传统人工方式下，我根据自己对模块的理解手动设计测试用例。设计时主要从“模块是否能够正常运行”的角度出发，重点考虑了摄像头正常读取、无人脸、多人脸和字段完整性等场景。

人工设计过程包括以下步骤：

- 1. 回顾模块功能；
- 2. 查看输出字段；
- 3. 根据正常运行流程设计测试；
- 4. 补充少量异常情况；
- 5. 将测试用例整理为表格。

6.2 人工生成的测试用例

测试编号	测试目标	输入条件	执行步骤	预期结果	优先级
TC-H-01	验证摄像头正常打开	摄像头可用	启动视频采集模块	能够正常读取视频帧	高
TC-H-02	验证单人脸场景	画面中只有一名学生	读取视频帧并处理	faces 中包含一个人脸区域	高
TC-H-03	验证无人脸场景	画面中没有人脸	读取视频帧并处理	faces 为空列表，程序不崩溃	高
TC-H-04	验证多人脸场景	画面中出现多个人脸	读取视频帧并处理	faces 中包含多个人脸信息	中
TC-H-05	验证字段完整性	摄像头正常输入	检查输出结果	输出包含 timestamp 、 frame 、 faces 等字段	高
TC-H-06	验证空帧情况	视频流偶尔返回空帧	调用预处理流程	程序能够跳过或提示异常	中

6.3 人工方式结果统计

指标	结果
设计耗时	约25分钟
测试用例数量	6条
正常场景数量	2类
异常场景数量	2类
边界场景数量	1类
性能场景数量	0类
是否形成规范表格	是
是否需要后续补充	是

6.4 人工方式分析

传统人工方式的优点是设计者对项目实际情况比较熟悉，生成的测试用例基本符合当前模块功能，不会明显偏离项目范围。但缺点也比较明显：测试场景数量较少，主要集中在正常功能和少量异常情况，对性能场景、字段类型、边界坐标、连续帧稳定性等考虑不足。

这说明仅依靠人工经验设计测试用例，容易受到个人经验和时间限制影响，测试覆盖不够全面。

七、AI辅助方式验证过程与结果

7.1 AI辅助生成过程

AI辅助方式下，我将前文中的提示词输入AI工具。AI根据模块说明和接口字段，自动生成了测试用例表。生成结果中不仅包含正常场景，还补充了摄像头不可用、空帧、字段缺失、 face_roi 越界、连续多帧处理、帧率过低等人工方式中没有充分考虑的场景。

AI生成后，我对结果进行了人工筛选。筛选原则如下：

- 1. 与视频采集与预处理模块直接相关的用例保留；
- 2. 明显超出课程项目范围的用例删除；
- 3. 表述过于笼统的用例进行修改；
- 4. 重复用例进行合并；
- 5. 不可执行的用例不采纳。

7.2 AI辅助生成并筛选后的测试用例

测试编号	测试目标	输入条件	执行步骤	预期结果
TC-AI-01	验证摄像头正常读取	摄像头可用，权限正常	启动模块并读取视频流	成功输出 frame 和 timestamp
TC-AI-02	验证单人脸场景	画面中只有一名学生	读取一帧并进行预处理	faces 中包含1个元素，包含 face_id 和 face_roi
TC-AI-03	验证无人脸场景	画面中没有人脸	读取并处理当前帧	faces 为空列表， owner_face_id 为None或空值
TC-AI-04	验证多人脸场景	画面中出现多个人脸	读取并处理当前帧	faces 包含多个元素，系统能按规则处理主体人脸
TC-AI-05	验证摄像头权限异常	摄像头不可用或权限关闭	启动视频采集模块	系统输出错误提示或异常状态，不直接崩溃
TC-AI-06	验证空帧处理	视频流返回空帧	调用预处理函数	系统跳过该帧或返回异常状态

测试编号	测试目标	输入条件	执行步骤	预期结果
TC-AI-07	验证输出字段完整性	摄像头正常输入	检查模块输出数据结构	输出包含 timestamp 、 frame 、 faces 、 owner_face
TC-AI-08	验证 face_roi 坐标合法性	检测到人脸区域	检查 face_roi 坐标范围	坐标不超出图像边界，越界时能处理
TC-AI-09	验证 owner_face_id 为空情况	无法确定主体人脸	输出处理结果	owner_face_id 允许为空，但字段必须存在
TC-AI-10	验证 frame 格式一致性	输入正常图像帧	检查输出 frame 格式	frame 格式与接口说明一致
TC-AI-11	验证连续帧稳定性	连续读取100帧	运行模块并记录结果	程序不异常退出，输出结构保持稳定
TC-AI-12	验证低帧率情况	视频流帧率较低	启动模块并观察处理结果	系统能够继续运行并记录帧率异常
TC-AI-13	验证字段类型正确性	正常视频输入	检查各字段类型	timestamp 为float， faces 为list， frame 为ndarray
TC-AI-14	验证异常日志记录	输入异常视频流	运行模块	系统能够记录异常信息，便于后续排查

7.3 AI辅助方式结果统计

指标	结果
AI生成初稿耗时	约3分钟
人工筛选和修改耗时	约7分钟
总耗时	约10分钟
初始生成测试用例数量	16条
筛选后保留测试用例数量	14条
正常场景数量	3类
异常场景数量	5类
边界场景数量	3类
性能场景数量	2类
是否形成规范表格	是
是否需要人工修改	是

7.4 AI辅助方式分析

AI辅助方式在测试用例数量和场景覆盖范围上明显优于传统人工方式。AI不仅生成了正常流程测试，还补充了摄像头权限异常、空帧处理、字段类型检查、 face_roi 坐标合法性、连续帧稳定性和低帧率情况等场景。这些内容是人工方式中容易遗漏的。

但是，AI生成结果并不能直接使用。部分测试用例表述比较笼统，需要人工修改；部分测试场景虽然理论上合理，但需要结合项目实际实现情况判断是否可执行。因此，AI辅助测试用例生成更适合作为“测试设计初稿生成工具”，最终测试用例仍然需要人工确认。

八、对比数据分析

8.1 效率对比

对比项	传统人工方式	AI辅助方式	改进情况
总耗时	25分钟	10分钟	减少15分钟
测试用例数量	6条	14条	增加8条
异常场景数量	2类	5类	增加3类
边界场景数量	1类	3类	增加2类
性能场景数量	0类	2类	增加2类

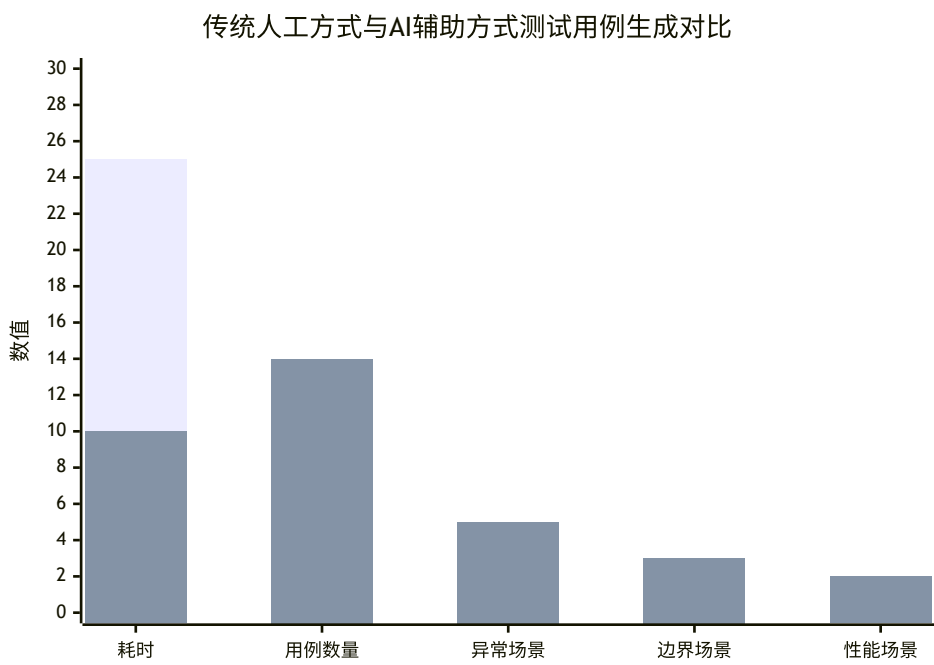
从耗时来看，AI辅助方式总耗时约10分钟，相比传统人工方式的25分钟减少了约60%。从测试用例数量来看，AI辅助方式筛选后保留14条测试用例，是人工方式6条的两倍以上。

8.2 质量对比

质量维度	传统人工方式	AI辅助方式
正常场景覆盖	较好	较好
异常场景覆盖	一般	较好
边界场景覆盖	较少	较全面
性能场景覆盖	基本没有	有一定覆盖
表格规范性	较规范	较规范
与项目相关性	高	较高，但需筛选
可直接使用程度	较高	中等，需要人工修改

从质量来看，人工方式的测试用例更贴近当前个人理解，但覆盖面较窄；AI辅助方式覆盖面更广，但需要人工筛选和修改。两者结合后，可以得到更高质量的测试用例。

8.3 可视化对比



说明：上图中，“耗时”的单位为分钟，其余指标为数量。可以看出，AI辅助方式在用例数量、异常场景、边界场景和性能场景方面都有提升，同时总耗时明显减少。

九、AI输出质量评价

9.1 可用内容

AI生成的以下内容具有较高可用性：

1. 正常摄像头读取测试；
2. 无人脸测试；
3. 多人脸测试；
4. 摄像头权限异常测试；
5. 空帧测试；
6. 字段完整性测试；
7. 字段类型测试；
8. 连续帧稳定性测试；
9. 低帧率测试；
10. 异常日志记录测试。

这些测试内容与视频采集与预处理模块高度相关，能够直接用于测试设计文档。

9.2 需要人工修改的内容

AI生成内容中也存在一些需要人工修改的地方：

1. 部分测试步骤过于笼统，需要补充具体执行方式；
2. 部分预期结果需要结合实际代码进行调整；
3. 部分测试用例存在重复，需要合并；
4. 部分性能测试指标没有具体阈值，需要人工根据项目情况确定；
5. 部分测试场景理论上合理，但当前项目未实现相关功能，因此不能直接使用。

9.3 不适合直接采纳的内容

AI有时会提出超出课程项目范围的测试内容，例如：

- 大规模并发视频流测试；
- 云端监控服务测试；
- 自动扩容测试；
- 长时间线上运行稳定性测试；
- 多终端分布式部署测试。

这些测试适合企业级系统，但对于本课程项目来说过重，因此不应采纳。

十、验证结论

通过本次最小可行验证，可以得出以下结论。

第一，AI辅助测试用例生成能够明显提高测试设计效率。传统人工方式设计6条测试用例耗时约25分钟，而AI辅助方式在人工筛选后得到14条测试用例，总耗时约10分钟，效率提升较明显。

第二，AI能够补充人工容易遗漏的异常场景和边界场景。人工方式主要关注摄像头正常读取、单人脸、无人脸等基本情况，而AI辅助方式进一步补充了摄像头权限异常、空帧、字段类型、face_roi 越界、连续帧稳定性和低帧率等场景。

第三，AI生成结果不能直接作为最终测试文档。虽然AI生成速度快、覆盖范围广，但部分内容需要人工筛选、修改和确认。AI可能生成过重、重复或与项目实际不完全匹配的测试场景。

第四，AI辅助方式最适合与人工方式结合使用。较合理的流程是：先由AI根据需求和接口说明生成测试用例初稿，再由测试人员根据项目实际情况进行筛选和修改，最后执行测试并记录结果。

因此，本次验证说明，在“面向网课的智能监督系统”中引入AI辅助测试用例生成具有较强可行性和实际价值。该改进可以纳入改进后的软件过程，作为集成与测试阶段的重要辅助活动。

十一、改进建议

根据本次验证结果，后续在项目中使用AI辅助测试时，可以从以下几个方面进一步改进。

11.1 优化提示词

AI生成测试用例的质量与提示词密切相关。后续可以在提示词中进一步明确：

- 1. 项目是课程小组项目，不需要企业级复杂测试；
- 2. 测试对象是视频采集与预处理模块；
- 3. 测试用例必须可执行；
- 4. 每条测试用例必须包含输入条件和预期结果；
- 5. 不生成与当前模块无关的测试。

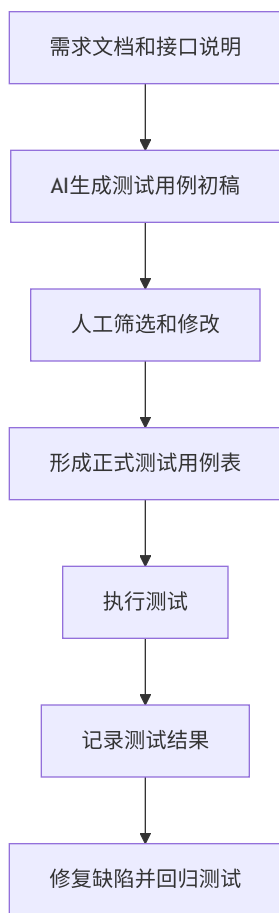
11.2 建立测试用例筛选标准

AI生成测试用例后，应按照以下标准筛选：

筛选标准	说明
相关性	是否与当前模块直接相关
可执行性	是否能在当前项目环境下执行
必要性	是否有助于发现实际问题
不重复性	是否与已有测试用例重复
适度性	是否适合课程项目规模

11.3 将AI辅助测试纳入正式过程

在改进后的软件过程中，可以将AI辅助测试用例生成固定为测试阶段的一项活动。具体流程为：



11.4 保留人工责任

AI辅助测试不能替代测试人员。测试人员仍需负责：

1. 判断测试用例是否合理；
2. 修改不符合项目实际的内容；
3. 执行测试；
4. 记录真实结果；
5. 分析缺陷原因；
6. 确认问题是否修复。

11.5 结合自动化测试工具

如果后续时间允许，可以进一步将AI生成的测试用例转化为PyTest测试脚本，实现部分自动化测试。例如，可以对输出数据结构进行字段完整性检查、类型检查和异常输入测试。这样可以进一步提高测试执行效率。

十二、总结

本报告围绕“AI辅助测试用例生成”场景，对“面向网课的智能监督系统”中的视频采集与预处理模块进行了最小可行验证。验证结果表明，AI辅助方式在测试用例生成效率、异常场景覆盖和边界场景补充方面具有明显优势。

传统人工方式虽然更贴近开发人员对项目的理解，但受经验和时间限制，测试用例数量较少，容易遗漏异常场景。AI辅助方式能够快速生成较多测试用例，并补充摄像头异常、空帧、字段类型、坐标越界、低帧率等场景，但也存在需要人工筛选和修改的问题。

因此，本项目中较合理的做法是采用“AI生成初稿 + 人工审核修改 + 实际执行验证”的模式。AI负责提高测试设计效率和补充测试思路，人工负责判断测试用例是否符合项目实际，并对测试结果承担最终责任。

整体来看，本次验证证明了AI技术可以较好地融入课程项目的软件过程，特别是在测试验证阶段具有较强的落地价值。这也为后续撰写“完整软件过程定义文档”提供了依据：在改进后的软件过程中，应正式加入AI辅助测试用例生成节点，并设置相应的质量标准、角色职责和度量指标。

